Class Notes: DAGs & Operators (Core Building Blocks)

What is a DAG?

A **D**irected **A**cyclic **G**raph (DAG) is the fundamental concept in Airflow. It is a collection of all the tasks you want to run, organized to reflect their relationships and dependencies.

- **Directed:** This means that tasks have a specific order. Relationships are one-way, represented as arrows (task_a >> task_b). You can't have a circular reference where a task points back to itself, either directly or through other tasks.
- **Acyclic:** This is the most important part. It means there can be no loops in the graph. A task can never depend on itself, either directly or indirectly. This prevents infinite loops in your workflow.
- **Graph:** It's a visual structure of nodes (tasks) and edges (dependencies).

In Airflow, a DAG is defined in a Python script. It doesn't perform any computation itself; it simply describes *how*, *when*, and *in what order* a set of tasks should be executed.

Key Properties of a DAG Object:

- dag_id: A unique identifier for the DAG.
- schedule_interval: Defines how often the DAG runs (e.g., @daily, None for manually triggered, or a cron expression like '0 3 * * *').
- start_date: The date from which the scheduler should begin scheduling DAG Runs.
- catchup: A boolean that determines if the scheduler should backfill missed DAG Runs between the start_date and the current date. Usually set to False to avoid accidental large backfills.
- default_args: A dictionary of arguments that are automatically passed to all operators within the DAG (e.g., retries, email_on_failure).
- tags: Labels to help categorize and filter your DAGs in the UI.

.

What is an Operator?

While a DAG describes *how* to run a workflow, an **Operator** defines what *actually* gets done. It is a class that encapsulates a single, atomic task.

- **Atomicity:** An operator should ideally perform one single task. For example, running a SQL query, executing a Bash script, or calling a Python function. If a task fails, it can be retried independently.
- **Templates:** Operators often use Jinja templating to dynamic parameters. Airflow provides built-in macros like {{ ds }} (the logical date as 'YYYY-MM-DD') that can be used in strings. For example, a BashOperator could run echo "Processing data for {{ ds }}".

- **Types of Operators:**
    - **Action Operators:** Perform an action.
        - BashOperator: Executes a bash command.
        - PythonOperator: Calls a Python function.
        - EmailOperator: Sends an email.
    - **Transfer Operators:** Move data from one system to another.
        - S3ToRedshiftOperator: Moves data from S3 to Redshift.
    - **Sensor Operators:** A special kind of operator that waits for something to happen.

Key Idea: You define the *workflow* (the DAG) and the *tasks* (the Operators) in Python code. Airflow then takes care of the scheduling, execution, and monitoring.

---

3. Providers

In early versions of Airflow, all operators for every service (AWS, Google, Snowflake, etc.) were bundled into the main project. This was messy and hard to maintain.

Providers are a modular system introduced to solve this. They are separate, installable Python packages that contain operators, hooks, sensors, and transfers for a specific service.

- **Example:** To use a PostgresOperator, you need to install the apache-airflow-providers-postgres package.
- **Separation of Concerns:** This keeps the Airflow core lightweight and allows provider teams to release updates on their own schedule.
- **Discovery:** You can browse all available providers on the Apache Airflow Providers page.

Command: pip install apache-airflow-providers-postgres

---

4. Create a Table

This is a practical task we would perform. We need a destination for our data. We would use a PostgresOperator to run a SQL CREATE TABLE statement.

Code Example:

```
from airflow.providers.postgres.operators.postgres import PostgresOperator


create_table = PostgresOperator(
  task_id="create_users_table",
  postgres_conn_id="postgres_default", # This references a connection defined in the UI
  sql="""
```

```
    CREATE TABLE IF NOT EXISTS users (
        firstname TEXT NOT NULL,
        lastname TEXT NOT NULL,
        country TEXT NOT NULL,
        username TEXT NOT NULL,
        password TEXT NOT NULL,
        email TEXT NOT NULL
    );
  """,
  dag=dag
)
```

---

5. Create a Connection

Operators need to know *how* to connect to external systems (databases, APIs, cloud platforms). We don't hardcode passwords or API keys in our DAG files. Instead, we store them securely in the Airflow **Metadata Database** as **Connections**.

How to create a connection in the UI:

1. Go to **Admin -> Connections**.
2. Click the **+** button to add a new connection.
3. Fill in the details:
    - **Conn Id:** A unique identifier you will use in your operators (e.g., postgres_default).
    - **Conn Type:** The type of system (e.g., Postgres, HTTP, AWS, Snowflake).
    - **Host:** The network address of the system.
    - **Schema:** The database name (for databases).
    - **Login:** Username.
    - **Password:** The password.
    - **Port:** The connection port.

Image Suggestion: A screenshot of the Airflow UI's "Add Connection" form, filled out with example details for a PostgreSQL database connection.

---

6. What is a Sensor?

A **Sensor** is a special type of Operator that waits for a certain condition to be true before it succeeds. This allows a workflow to pause until an external event occurs.

- **Purpose:** They are used to make DAGs reactive. Instead of running on a strict schedule, a DAG can wait for a file to arrive, a database partition to be updated, or an API to become available.
- mode **Parameter:** A critical setting for sensors.
    - mode='poke' (default): The sensor occupies a worker slot while it repeatedly checks the condition. Use this for conditions expected to be met quickly.
    - mode='reschedule': The sensor checks the condition and then frees up the worker slot, rescheduling itself to check again later. Use this for long waits to avoid blocking worker resources.
- timeout **and** poke_interval**:** Control how long and how often the sensor checks.

Example: FileSensor, HttpSensor, SqlSensor.

---

7. Create the connection user_api

Before we can extract data, we need a connection to the source API. This would be an HTTP-type connection.

- **Conn Id:** user_api
- **Conn Type:** HTTP
- **Host:** https://randomuser.me/ (Example API)
- We might not need a login/password if it's a public API.

---

8. Extract users

This task would use the SimpleHttpOperator or the HttpOperator from the http provider to call an API and pull data.

Code Example:

```
from airflow.providers.http.operators.http import SimpleHttpOperator


extract_users = SimpleHttpOperator(
  task_id='extract_users',
  http_conn_id='user_api', # Uses the connection we created
  endpoint='api/', # Appended to the base URL in the connection
  method='GET',
  response_filter=lambda response: response.json()["results"], # Processes the response
  log_response=True,
  dag=dag,
```

```
)
```

- response_filter**:** This is a powerful parameter. It's a function that processes the response from the HTTP request. In this case, it parses the JSON and extracts the "results" list.

---

9. Process users

The raw data from an API often needs to be transformed. This is a perfect job for the PythonOperator, which calls a custom Python function.

Code Example:

```
from airflow.operators.python import PythonOperator


def _process_user(ti):
    """
    A function to process the raw user data.
    'ti' is the Task Instance, passed automatically by Airflow.
    """
    users = ti.xcom_pull(task_ids='extract_users') # Pull data from the 'extract_users' task

    if not users:
        raise ValueError("No users found in XCom")

    processed_users = []
    for user in users:
        processed_users.append({
            'firstname': user['name']['first'],
            'lastname': user['name']['last'],
            'country': user['location']['country'],
            'username': user['login']['username'],
            'password': user['login']['password'],
            'email': user['email']
        })

    # Push the processed data to XCom so the next task can use it
    ti.xcom_push(key='processed_users', value=processed_users)
```

```
process_users = PythonOperator(
    task_id='process_users',
    python_callable=_process_user, # Points to the function above
    dag=dag,
)
```

---

10. Before running process_user (XCom Explained)

The process_users task needs data from the extract_users task. How does it get it?
Through **XCom** (short for Cross-Communication).

- **What it is:** XCom is Airflow's mechanism for allowing tasks to exchange small amounts of data. It is stored in the Airflow metadata database.
- **How it works:**
    - A task can **push** data to XCom (e.g., ti.xcom_push(key='my_key', value=my_data)).
    - A downstream task can **pull** that data (e.g., ti.xcom_pull(task_ids='upstream_task_id', key='my_key')).
- **Limitation:** XCom is **not** designed for large data sets (e.g., multi-gigabyte files). For large data, use an intermediate storage like S3, HDFS, or a database, and only pass the *path* to the data via XCom.

---

11. What is a Hook?

We used a PostgresOperator to create a table. But what if we need more flexibility, like writing data from a Python function? This is where **Hooks** come in.

- **Hooks are abstractions for API and database connections.** They provide a programmatic, Pythonic interface to interact with external systems.
- **Hooks vs. Operators:**
    - An **Operator** is a pre-defined *task*. It uses a Hook under the hood but is configured for a specific action.
    - A **Hook** is a *building block*. It manages the connection and provides methods (like run(), get_records(), insert_rows()) that you can use inside a PythonOperator for maximum flexibility.
- **Why use a Hook?** When no existing operator does exactly what you need.

---

12. Store users

This task will take the processed user data and store it in the PostgreSQL table. We'll use a PythonOperator with a **Postgres Hook** for maximum control.

Code Example:

```
from airflow.providers.postgres.hooks.postgres import PostgresHook


def _store_users(ti):
    """
    A function to store processed user data in PostgreSQL.
    """
    processed_users = ti.xcom_pull(task_ids='process_users', key='processed_users')

    if not processed_users:
        raise ValueError("No processed users found in XCom")

    # Create a Postgres Hook using the 'postgres_default' connection
    pg_hook = PostgresHook(postgres_conn_id='postgres_default')

    # Define the INSERT query. Use %s as placeholders for parameters.
    insert_query = """
        INSERT INTO users (firstname, lastname, country, username, password, email)
        VALUES (%s, %s, %s, %s, %s, %s)
    """

    # Prepare the list of tuples for the executemany operation
    rows_to_insert = [
        (user['firstname'], user['lastname'], user['country'], user['username'], user['password'],
user['email'])
        for user in processed_users
    ]

    # Use the hook to run the query, inserting all rows at once
    pg_hook.insert_rows(
        table='users',
```

```
        rows=rows_to_insert,
        target_fields=['firstname', 'lastname', 'country', 'username', 'password', 'email']
    )

store_users = PythonOperator(
    task_id='store_users',
    python_callable=_store_users,
    dag=dag,
)
```

---

13. DAG Scheduling

Finally, we define the DAG itself and, most importantly, the **task dependencies**. This tells Airflow the order of operations.

Code Example:

```
from airflow import DAG
from datetime import datetime

# Define the default arguments for the DAG
default_args = {
    'start_date': datetime(2023, 10, 27),
    'retries': 1,
}

# Instantiate the DAG object
with DAG(
    'user_processing', # dag_id
    default_args=default_args,
    schedule_interval='@daily',
    catchup=False,
    tags=['training']
) as dag:

    # This is where we would list all our task variables
```

```
    create_table_task = PostgresOperator(...)  # from step 4
    extract_users_task = SimpleHttpOperator(...) # from step 8
    process_users_task = PythonOperator(...) # from step 9
    store_users_task = PythonOperator(...) # from step 12

    # Define the dependencies using bitshift operators
    # This is the same as:
    # create_table_task.set_downstream(extract_users_task)
    # extract_users_task.set_downstream(process_users_task)
    # etc.
    create_table_task >> extract_users_task >> process_users_task >> store_users_task

# The 'with' statement and dependency definition above are crucial.
```